Loops in C (Repetition Statements)

1. Introduction

In programming, a **loop** is used to repeat a block of code multiple times.

Sometimes we want a program to perform the **same task repeatedly** without writing the same code many times.

Example situations where loops are useful:

- Printing numbers from **1 to 100**
- Calculating the **sum of many numbers**
- Processing **students' marks**
- Displaying items in a **list**

Instead of writing the same instruction many times, we use **loops**.

Example Without a Loop

```
printf("Hello\n");
printf("Hello\n");
printf("Hello\n");
printf("Hello\n");
printf("Hello\n");
```

This is inefficient.

Example Using a Loop

```
for(int i = 1; i <= 5; i++)
{
   printf("Hello\n");
}
```

The loop prints **Hello five times** using only a few lines of code.

Types of Loops in C

C provides **three main loops**:

1. for loop
2. while loop
3. do...while loop

1. The for Loop

The **for loop** is used when the number of repetitions is **known in advance**.

Syntax

```
for(initialization; condition; update)
{
   statements;
}
```

Components

Part	Meaning
Initialization	Starting value
Condition	Loop continues while true
Update	Changes the loop variable

Flow of a For Loop

1. Initialization executes once.
2. Condition is checked.
3. If true → execute statements.
4. Update executes.
5. Condition checked again.
6. Loop continues until condition becomes **false**.

Example 1: Print Numbers 1 to 10

```
#include <stdio.h>

int main()

{

   int i;


   for(i = 1; i <= 10; i++)

   {

      printf("%d\n", i);

   }


   return 0;

}
```

Output

```
1

2

3

4

5

6

7

8

9

10
```

Example 2: Print Even Numbers

```
for(int i = 2; i <= 20; i = i + 2)
{
   printf("%d\n", i);
}
```

Output:

2 4 6 8 10 12 14 16 18 20

2. The while Loop

The **while loop** repeats a block of code **while a condition remains true**.

It is useful when the number of repetitions is **not known beforehand**.

Syntax

```
while(condition)
{
   statements;
}
```

The condition is tested **before the loop executes**.

Example: Print Numbers 1 to 5

```
#include <stdio.h>

int main()
{
   int i = 1;

   while(i <= 5)
   {
```

```
        printf("%d\n", i);
        i++;
    }

    return 0;
}
```

How It Works

1. i starts at **1**
2. Condition i <= 5 is checked
3. If true → print value
4. i increases
5. Loop continues until condition becomes **false**

3. The do...while Loop

The **do-while loop** is similar to the while loop but with one key difference:

The loop body **always executes at least once**, even if the condition is false.

Syntax

```
do
{
    statements;
}
while(condition);
```

Example

```
#include <stdio.h>

int main()
{
   int i = 1;

   do
   {
      printf("%d\n", i);
      i++;
   }
   while(i <= 5);

   return 0;
}
```

Key Difference Between Loops

Loop Type	Condition Checked
for	Before execution
while	Before execution
do-while	After execution

Comparison of Loops

Feature	for	while	do-while
Used when	Repetitions known	Repetitions unknown	Must run at least once
Condition check	Beginning	Beginning	End

Loop Control Statements

Sometimes we need to **change the normal flow of a loop**.

C provides special statements for this.

1. break
2. continue

1. break Statement

The break statement **immediately terminates the loop**.

Example

```
for(int i = 1; i <= 10; i++)
{
   if(i == 5)
   {
      break;
   }

   printf("%d\n", i);
}
```

Output

1

2

3

4

The loop stops when i = 5.

2. continue Statement

The continue statement **skips the current iteration** and moves to the next loop cycle.

Example

```
for(int i = 1; i <= 5; i++)
{
   if(i == 3)
   {
      continue;
   }

   printf("%d\n", i);
}
```

Output

1

2

4

5

Number **3 is skipped**.

Infinite Loops

An **infinite loop** occurs when the condition never becomes false.

Example:

```
while(1)
{
   printf("Hello");
}
```

This loop runs **forever**.

Example: Sum of Numbers 1 to 10

```
#include <stdio.h>


int main()
{
   int i, sum = 0;


   for(i = 1; i <= 10; i++)
   {
      sum = sum + i;
   }


   printf("Sum = %d", sum);


   return 0;
}
```

Output

Sum = 55